2

Introduction to C++ Programming

OBJECTIVES

- To write simple computer programs in C++.
- To write simple input and output statements.
- To use fundamental types.
- To describe the basic concept of computer memory.
- To use arithmetic operators.
- To write simple decision-making statements.

2.1 Introduction

- **Five examples demonstrate**
  - **How to display messages**
  - **How to obtain input data from the user**
  - **How to perform arithmetic calculations**
  - **How to make decisions**
  - **How to compare integers**

2.2 First Program in C++: Printing a Line of Text

- **Example 1: Print a line of text.**
  - **Illustrates several important features of C++.**
  - **Note that the number on the left hand side is the line number.**

```cpp
// Fig. 2.1: fig02_01.cpp
// Text-printing program.
#include <iostream> // allows program to output data to the screen

// function main begins program execution
int main()
{
   std::cout << "Welcome to C++!\n"; // display message

   return 0; // indicate that program ended successfully

} // end function main
```

```
Welcome to C++!
```

Outline

fig02_01.cpp
(1 of 1)

fig02_01.cpp
output (1 of 1)

2.2 First Program in C++: Printing a Line of Text

```
Line 1: // Fig. 2.1: fig02_01.cpp
Line 2: // Text-printing program.
```

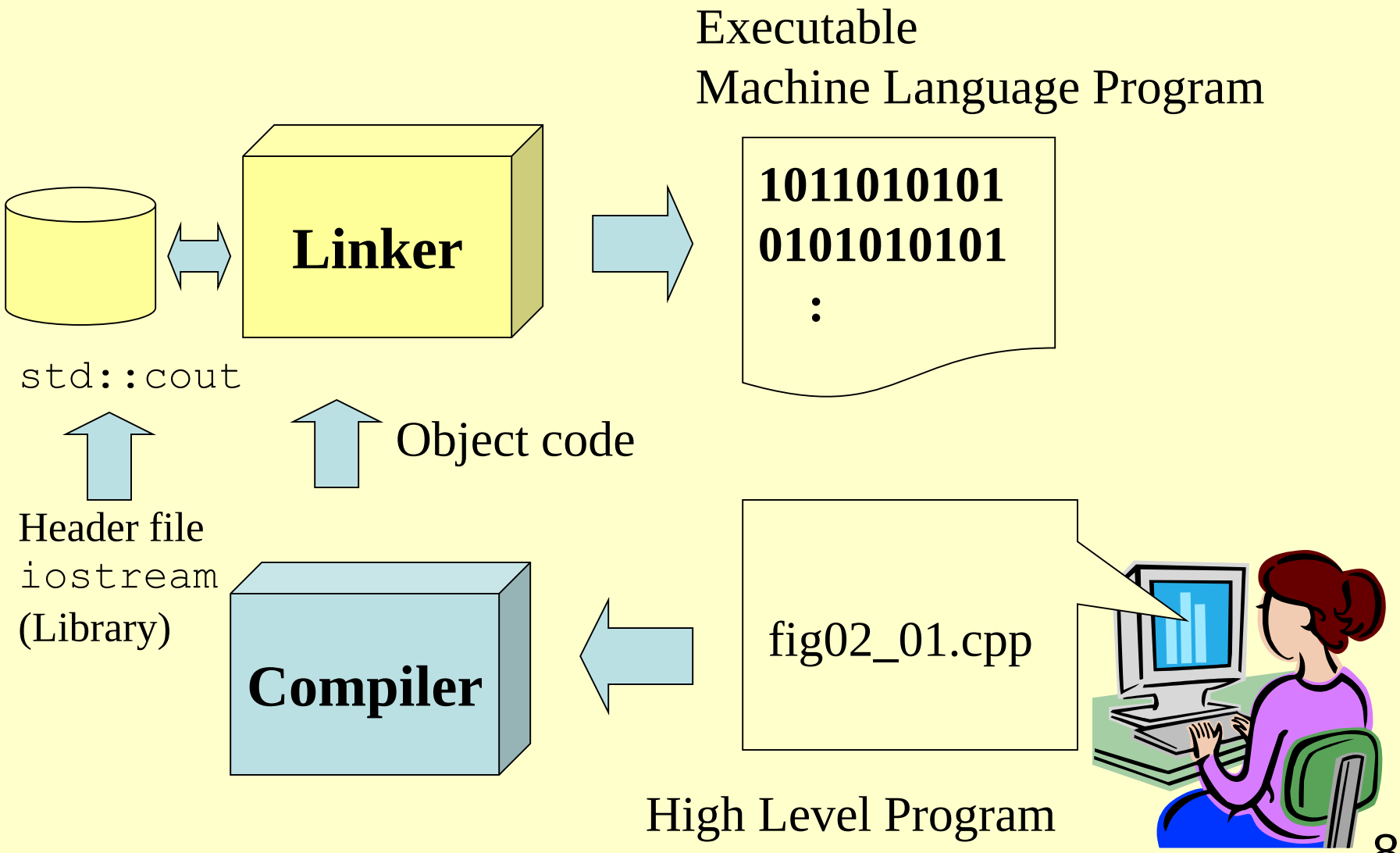
- **Comments**
  - **Readable for programmers**
  - **Ignored by compiler**
  - **Single-line comment: Begin with `//`**
  - **Multi-line comment: Start with `/*` and end with `*/`**

2.2 First Program in C++: Printing a Line of Text

Line 3: #include <iostream>

- **Preprocessor directive**
  - **`#include` inserts the header file `iostream` to the program.**
  - **Later, we can use the function `cout` from the header `iostream`.**

2.2 First Program in C++: Printing a Line of Text



2.2 First Program in C++: Printing a Line of Text

- **Two ways to include a file:**
  - **`#include <iostream>` : get the header file `iostream` through the default path.**
  - **`#include "iostream"` : get the header file `iostream` in the current directory.**
    - **Current directory = the directory contain the C++ program file (e.g., fig02_01.cpp).**

2.2 First Program in C++: Printing a Line of Text

Line 4:

- **White space**
  - **Readable for programmers**
  - **Ignored by the compiler**
  - **Also for space characters and tabs**

2.2 First Program in C++: Printing a Line of Text

Line 6: int main()

- **Function `main`**
  - **C++ programs begin executing at function `main`.**
  - **Can "return" a value**
    - **Example: This `main` function returns an integer (whole number).**

2.2 First Program in C++: Printing a Line of Text

Line 7: {

…

Line 12: }

- **The function `main` is delimited by braces (`{}`).**
- **Multiple statements created inside a pair of braces are called a compound statement or a block.**

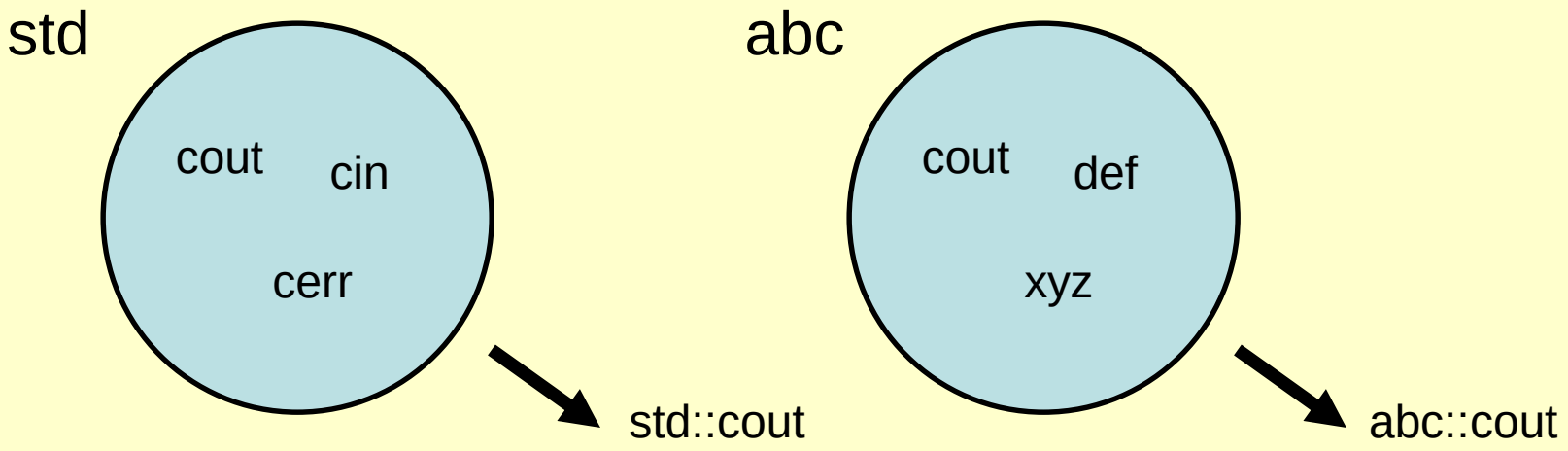
2.2 First Program in C++: Printing a Line of Text

```
Line 8: std::cout << “Welcome to C++!\n”;
```

- **Statements**
  - **Instruct the program to perform an action.**
  - **All statements end with a semicolon (;).**

2.2 First Program in C++: Printing a Line of Text

- **Namespaces `std::`**
  - **Specifies using a name that belongs to "namespace" `std`.**
  - **Namespace is the name of a group.**
  - **It is used to avoid identifier overlapping.**



2.2 First Program in C++: Printing a Line of Text

- `std` = Standard output stream object
  - `std::cout`
    - "Connect" to the screen.
    - Define in the header file `<iostream>`.

2.2 First Program in C++: Printing a Line of Text

- **Stream insertion operator <<**
  - **Value to right (right operand) inserted into left operand.**
  - **Example**
    - `std::cout << "Hello";`
      - **Inserts the string `"Hello"` into the standard output.**
      - **i.e., Display to the screen.**

2.2 First Program in C++: Printing a Line of Text

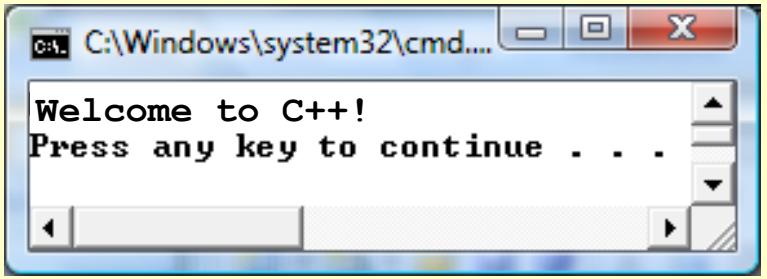
- **Escape character \n**
  - **Newline.**
  - **Position the screen cursor to the beginning of the next line.**
  - **"\" indicates special character output.**
  - **Others**
    - **\t – Horizontal tab**
    - **\r – Carriage return**
    - **\a – Alert**

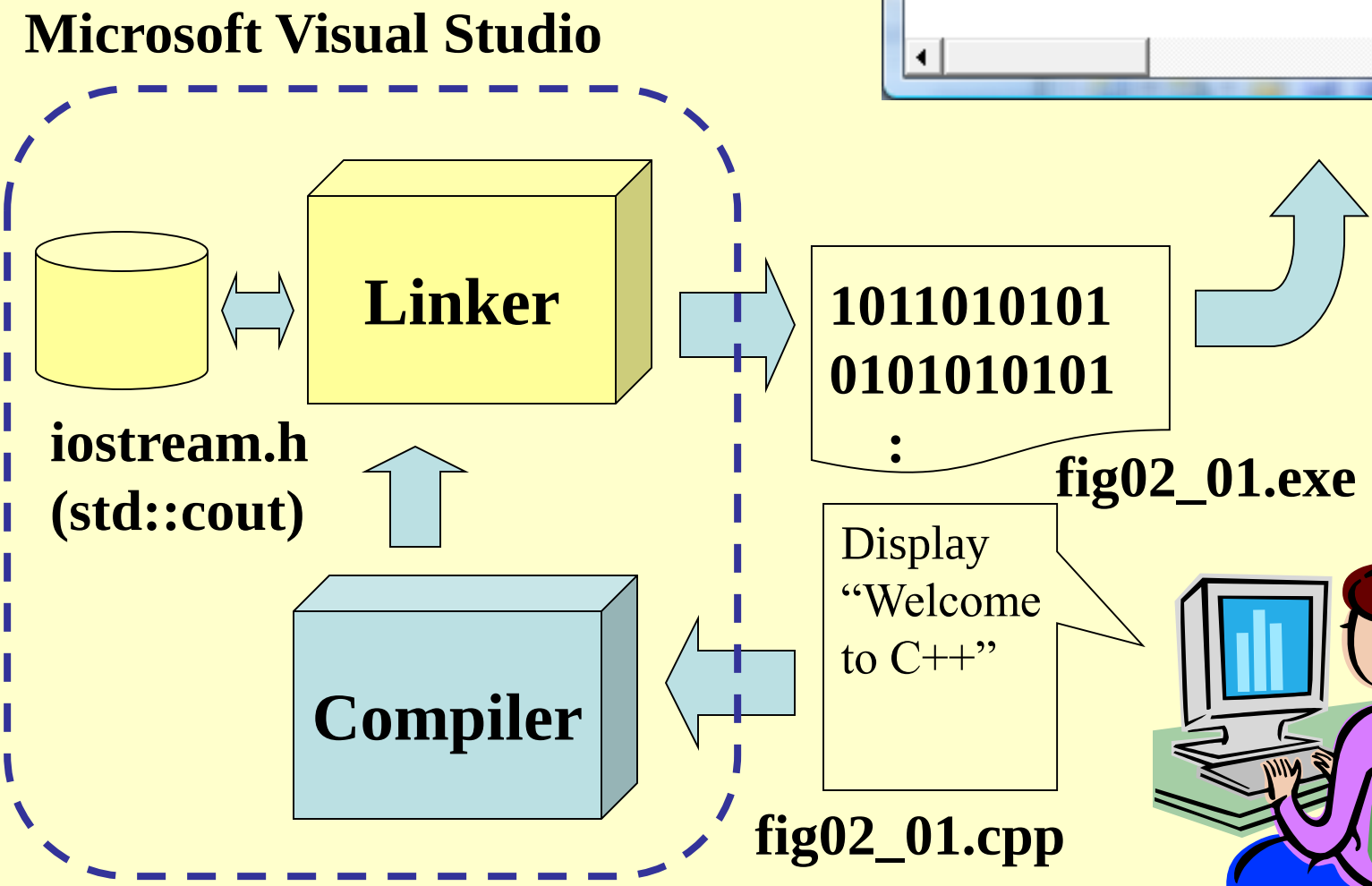
2.2 First Program in C++: Printing a Line of Text

Line 10: return 0;

- **`return` statement**
  - **Used at the end of `main`.**
  - **The value `0` indicates the program terminated successfully.**

2.2 First Program in C++: Printing a Line of Text





2.3 Modifying Our First C++ Program

- **Example 2: Print text on one line using multiple statements.**
  - **Each stream insertion resumes printing where the previous one stopped.**

```cpp
// Fig. 2.3: fig02_03.cpp
// Printing a line of text with multiple statements.
#include <iostream> // allows program to output data to the screen

// function main begins program execution
int main()
{
   std::cout << "Welcome ";
   std::cout << "to C++!\n";

   return 0; // indicate that program ended successfully

} // end function main
```

```
Welcome to C++!
```

Outline

fig02_03.cpp

(1 of 1)

fig02_03.cpp output (1 of 1)

2.3 Modifying Our First C++ Program

- **Example 3: Print text on several lines using a single statement.**
  - **Each newline escape sequence positions the cursor to the beginning of the next line.**
  - **Two newline characters back to back outputs a blank line.**

```cpp
// Fig. 2.4: fig02_04.cpp
// Printing multiple lines of text with a single statement.
#include <iostream> // allows program to output data to the screen

// function main begins program execution
int main()
{
   std::cout << "Welcome\nto\n\nC++!\n";

   return 0; // indicate that program ended successfully

} // end function main
```

```
Welcome
to

C++!
```

Outline

fig02_04.cpp

(1 of 1)

fig02_04.cpp output (1 of 1)

2.4 Another C++ Program: Adding Integers

- **Example 4: Adding integers**
  - **Get two integers from users.**
  - **Add two integers.**
  - **Display the result to the screen.**

```
1  // Fig. 2.5: fig02_05.cpp
2  // Addition program that displays the sum of two numbers.
3  #include <iostream> // allows program to perform input and output
4
5  // function main begins program execution
6  int main()
7  {
8     // variable declarations
9     int number1; // first integer to add
10    int number2; // second integer to add
11    int sum; // sum of number1 and number2
12
13    std::cout << "Enter first integer: "; // prompt user for data
14    std::cin >> number1; // read first integer from user into number1
15
16    std::cout << "Enter second integer: "; // prompt user for data
17    std::cin >> number2; // read second integer from user into number2
18
19    sum = number1 + number2; // add the numbers; store result in sum
20
21    std::cout << "Sum is " << sum << std::endl; // display sum; end line
22
23    return 0; // indicate that program ended successfully
24
25 } // end function main
```

```
Enter first integer: 45
Enter second integer: 72
Sum is 117
```

Outline

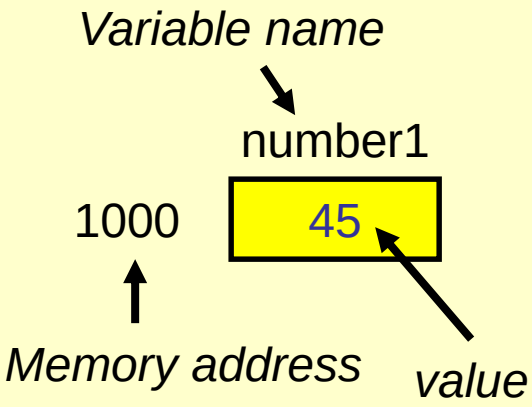
fig02_05.cpp

(1 of 1)

fig02_05.cpp output (1 of 1)

2.4 Another C++ Program: Adding Integers

```
Line 9:   int number1;
Line 10:  int number2;
Line 11:  int sum;
```



- **Declaration**
  - **`number1`, `number2` and `sum` are variables.**
  - **Variable: Location in computer's memory where a value can be stored for use by a program.**
  - **Declare variables with name and data type before use.**
    - `int` **– integer numbers**
    - `char` **– characters**
    - `double` **– floating point numbers**

2.4 Another C++ Program: Adding Integers

- **Multiple declarations**
  - `int integer1, integer2, sum;`
- **Variable name**
  - **Series of characters (letters, digits, underscores)**
  - **Cannot begin with digit**
  - **Case sensitive**
- **Variable size**
  - **Depends on the data type**
  - **Examples: `int` (4 bytes), `char` (1 byte), `double` (8 bytes).**

2.4 Another C++ Program: Adding Integers

```
Line 14: std::cin >> number1;
```

- **Input stream object**
  - **`std::cin` from `<iostream>`**
  - **Usually connect to the keyboard**
- **Stream extraction operator >>**
  - **Waits for user to input value, press *Enter* (Return) key**
  - **Stores value in variable to right of operator**

2.4 Another C++ Program: Adding Integers

```
Line 19: sum = number1 + number2;
```

- **Assignment operator =**
  - **Assigns value on left to variable on right**
  - **This statement adds the values of `number1` and `number2` and stores result in `sum`.**

2.4 Another C++ Program: Adding Integers

```
Line 21: std::cout << “Sum is “ << sum << std::endl;
```

- **Concatenating stream insertion operations**
  - **Use multiple stream insertion operators in a single statement.**
  - **This statement outputs `"Sum is “` first. Then, it outputs the value of `sum`.**
- **Stream manipulator `std::endl`**
  - **Outputs a newline.**
  - **Flushes the output buffer.**

Class Work

```
int abc;
…
abc = 2;
…
```

- **What is the name of the variable?**
- **What is the data type of the variable?**
- **What is the size of the variable?**
- **What is the value of the variable? 2**
- **What is the memory address of the variable?!**

2.6 Arithmetic

- **Arithmetic operators**
  - **`*` : Multiplication**
  - **`/` : Integer division truncates remainder**
    - **`7 / 5` evaluates to 1**
  - **`%` : Modulus operator returns remainder**
    - **`7 % 5` evaluates to 2**

2.6 Arithmetic

- **Parentheses are used in C++ expressions to group sub-expressions**
  - **Same manner as in algebraic expressions**
  - **Example: `a * ( b + c )`**
    - `b + c` **first and then multiple** `a`**.**
  - **Multiplication, division, modulus applied next**
  - **Addition, subtraction applied last**

2.7 Decision Making: Equality and Relational Operators

- **Condition**
  - **Expression can be either `true` or `false`**
- **`if` statement**
  - **If condition is `true`, body of the `if` statement executes**
  - **If condition is `false`, body of the `if` statement does not execute**

Standard algebraic equality or relational operator	C++ equality or relational operator	Sample C++ condition	Meaning of C++ condition
Relational operators			
>	>	x > y	x is greater than y
<	<	x < y	x is less than y
≥	>=	x >= y	x is greater than or equal to y
≤	<=	x <= y	x is less than or equal to y
Equality operators			
=	==	x == y	x is equal to y
≠	!=	x != y	x is not equal to y

Fig. 2.12 | **Equality and relational operators.**

2.7 Decision Making: Equality and Relational Operators

- **Example 5: Integer comparison**

Outline

fig02_13.cpp

(1 of 2)

```
1  // Fig. 2.13: fig02_13.cpp
2  // Comparing integers using if statements, relational operators
3  // and equality operators.
4  #include <iostream> // allows program to perform input and output
5
6  using std::cout; // program uses cout
7  using std::cin;  // program uses cin
8  using std::endl; // program uses endl
9
10 // function main begins program execution
11 int main()
12 {
13    int number1; // first integer to compare
14    int number2; // second integer to compare
15
16    cout << "Enter two integers to compare: "; // prompt user for data
17    cin >> number1 >> number2; // read two integers from user
18
19    if ( number1 == number2 )
20       cout << number1 << " == " << number2 << endl;
21
22    if ( number1 != number2 )
23       cout << number1 << " != " << number2 << endl;
24
25    if ( number1 < number2 )
26       cout << number1 << " < " << number2 << endl;
27
28    if ( number1 > number2 )
29       cout << number1 << " > " << number2 << endl;
30
```

```
31    if ( number1 <= number2 )
32       cout << number1 << " <= " << number2 << endl;
33
34    if ( number1 >= number2 )
35       cout << number1 << " >= " << number2 << endl;
36
37    return 0; // indicate that program ended successfully
38
39 } // end function main
```

```
Enter two integers to compare: 3 7
3 != 7
3 < 7
3 <= 7
```

```
Enter two integers to compare: 22 12
22 != 12
22 > 12
22 >= 12
```

```
Enter two integers to compare: 7 7
7 == 7
7 <= 7
7 >= 7
```

Outline

fig02_13.cpp

(2 of 2)

fig02_13.cpp output (1 of 3)

(2 of 3)

(3 of 3)

2.7 Decision Making: Equality and Relational Operators

```
Line 6: using std::cout;
Line 7: using std::cin;
Line 8: using std::endl;
```

- **`using` declarations**
  - **Eliminates the need to repeat the `std::` prefix.**
  - **A more convenient way is**

```
using namespace std;
```

  - **Enable a program to use all the names in any standard C++ header (e.g., <iostream>) that a program might include.**

2.7 Decision Making: Equality and Relational Operators

```
Line 17: cin >> number1 >> number2;
```

- **Read two numbers by using one `cin`**
  - **Two numbers must be separated by a space character.**
  - **The first value is read into variable `number1`, then a value is read into variable `number2`.**

Until Next Time

- **Review Chapter 2**
- **Preview Chapter 3**